Fawtir mobile: API reference, credit notes and debit notes

Oct 1, 2026 · @Rixon Agency

Every API the web app calls for sales credit notes and debit notes in Income, as it calls them today on `main`. A credit note (PINT-OM type 381) lowers what a customer owes on an issued invoice; a debit note (383) charges them more on it. Both always start from an issued invoice and carry its reference (IBR-032-OM).

Use it with two earlier docs: "Fawtir mobile: API reference, sign-up to issued invoice" for the shared conventions (base URL, sign-in, headers, idempotency, versions, errors, money), and "Fawtir mobile: credit notes and debit notes" for the screens and figures. Self-billed notes and supplier notes are out of scope.

Conventions

The same as the invoice API doc, section Conventions. In short:

- `Idempotency-Key: draft-<uuid>` on create and issue, `payment-<uuid>` on payments. Reuse the key after a timeout, network error or 5xx; make a new one only after a 4xx.
- `expectedVersion` in the body on every update, issue and cancel; `?expected_version=` on discard. `STALE_INVOICE_VERSION` means reload.
- Errors are problem JSON; the module code is in `code`. Field errors are `errors[]` with paths such as `draft.lines[1].quantity`.
- Three decimals, half up; currency `OMR` only, shown with the rial sign.
- Hide every action the user may not take.

1. Starting a note

A note always starts from an issued invoice: `type` `INVOICE` and `authoringState` `ISSUED`. Offer "Credit note" only with `creditnote.create` and "Debit note" only with `debitnote.create`.

#	Call	Answer used	When
1.1	`GET /v1/authored-invoices?state=ISSUED&type=INVOICE&q=&sort=ISSUED_NEWEST&page=0&size=20`	`id`, `invoiceNumber`, `customerName`, `totals.totalAmount`, `issueDate`	Only when no invoice was chosen: the invoice picker
1.2	`GET /v1/authored-invoices/{invoiceId}`	The invoice: `customerId`, `branchId`, `currency`, `details` (`paymentTerms`, `purchaseOrderReference`, `transactionType`), `headerDiscount`, `lines[]`	Always
1.3	`GET /v1/authored-invoices/{invoiceId}/receivable`	`documentTotal`, `paidAmount`, `creditedAmount`, `outstandingAmount`, `paymentState`, `overdue`	Always; if it fails, carry on without the balance figures
1.4	`GET /v1/customers/{customerId}`	`type`: a `CONSUMER` hides line classification	Only with `customer.read`
1.5	`GET /v1/invoice-drafts/{draftId}`	The saved note draft; the invoice is its `correctionOfInvoiceId`	Reopening a draft note, then 1.2 to 1.4

Figures from 1.3: creditable = `documentTotal` − `creditedAmount`. Credit: new balance = max(outstanding − credit, 0) and refund due = max(credit − outstanding, 0). Debit: new balance = outstanding + note total.

If the invoice is not issued or was cancelled, block Save and Issue with: "This invoice is not issued (or was cancelled), so it cannot be corrected."

2. Note draft: create to issued

Save draft needs `invoice.draft`; Issue needs `invoice.issue`. On a click the web runs: check the rules (section 3), create or update, upload queued files, then issue.

#	Call	Body	Answer	Notes
2.1	`POST /v1/authored-invoices/{invoiceId}/credit-notes`	Header `Idempotency-Key: draft-<uuid>`. Body = the note body below, **not wrapped**	The draft: `{id, version, authoringState, invoiceNumber, type, correctionOfInvoiceId, details, headerDiscount, lines[]}`	Only `POST /v1/invoice-drafts` wraps its body; this route does not
2.2	`POST /v1/authored-invoices/{invoiceId}/debit-notes`	Same as 2.1	Same as 2.1	

#	Call	Body	Answer	Notes
2.3	`PUT /v1/invoice-drafts/{id}`	`{expectedVersion, draft: <note body>}` (wrapped), no idempotency key	The draft; keep the new `version`	`STALE_INVOICE_VERSION`: reload
2.4	`GET /v1/invoice-drafts/{id}`	none	The draft	To reopen a note draft
2.5	`DELETE /v1/invoice-drafts/{id}?expected_version=<n>`	none	The draft is discarded	
2.6	`POST /v1/authored-invoices/{draftId}/attachments`	Multipart field `file`	`{id, filename, contentType, byteSize, sha256, attachedAt}`	Needs `invoice.attachment.manage`. Draft only. The web accepts `.pdf .jpg .jpeg .png .xlsx`
2.7	`POST /v1/invoice-drafts/{id}/issue`	Header `Idempotency-Key: draft-<uuid>`, a key separate from create. Body `{expectedVersion, issueDate, dueDate}`	The issued note; open its detail	Credit: `dueDate` = `issueDate`. Debit: issue date + the invoice terms' `netDays`, editable, not before the issue date

The web does not call preview (`POST /v1/invoice-drafts/{id}/preview`) for notes: it works the totals out on the device with the invoice math. Preview is there if mobile wants the server's figures.

Note body (unknown fields are refused):

```
{
  "customerId": "…", "branchId": "…", "currency": "OMR", "vatMode":
"EXCLUSIVE",
  "headerDiscount": { "type": "FIXED_AMOUNT", "value": 0 },
  "details": {
    "deliveryDate": null,
    "subject": "RMA-0142",
    "purchaseOrderReference": "PO-88213",
    "customerVisibleNote": "4 kg returned, quality issue",
    "paymentTerms": { "code": "NET30", "label": "Net 30 days" },
    "transactionType": "FULL_TAX",
    "correctionReasonCode": "QTY",
    "correctionReasonDetail": "returnedGoods"
  },
  "lines": [{
    "lineNumber": 1, "productId": "…", "productCode": "BEANS-H",
    "description": "Coffee beans 1 kg, house blend", "unitCode": "KGM",
    "quantity": 4, "unitPrice": 9.5,
```

```
    "discount": { "type": "PERCENTAGE", "value": 10 },
    "taxCategory": { "code": "S5", "en16931Category": "S", "rate": 5,
                     "exemptionReason": null, "exemptionReasonCode": null },
    "classification": { "goodsOrService": "G", "hsCode": "090121000000" }
  }]
}
```

Copied from the invoice, never asked: `customerId`, `branchId`, `currency`, `details.purchaseOrderReference`, `details.paymentTerms`, `details.transactionType` (`SIMPLIFIED` if the invoice was, else `FULL_TAX`). `vatMode` is always `EXCLUSIVE` and `deliveryDate` always `null`.

Entered on the note: `details.subject` = the "Reference / RMA" text (max 200) or `null`; `details.customerVisibleNote` (max 1,000) or `null`; the reason as `correctionReasonCode` (BTOM-032, CL-02-OM) plus `correctionReasonDetail` (the reason id). Leave out `internalNote`, `buyerReference` and `contractReference` (the last two are refused: `BUYER_REFERENCES_NOT_SELF_BILLED`).

Reasons

Note	Reason (sent as `correctionReasonDetail`)	Code
Credit	`returnedGoods` Returned goods	QTY
Credit	`priceCorrection` Price correction	VAL
Credit	`orderCancelled` Order cancelled	CAN
Credit	`vatCorrection` VAT correction	VAT
Credit	`goodwillDiscount` Goodwill discount	OTH
Credit	`other` Other	OTH
Debit	`undercharged` Undercharged	VAL
Debit	`additionalDelivery` Additional delivery	QTY
Debit	`freightOmitted` Freight omitted	OTH
Debit	`vatCorrection` VAT correction	VAT
Debit	`latePaymentCharge` Late payment charge	OTH
Debit	`priceEscalation` Price escalation	VAL

Note	Reason (sent as `correctionReasonDetail`)	Code
Debit	`other` Other	OTH

Reading back: use the stored `correctionReasonDetail` when it pairs with the code, else the first reason with that code (`OTH` reads as Other). Older servers may not return the detail yet.

3. Lines, modes, discount and VAT

What a note's `lines[]` and `headerDiscount` hold depends on the mode the user picks. Only included lines are sent, renumbered from 1.

Note	Mode	`lines[]` sent	`headerDiscount` sent
Credit	Full invoice (default)	Every invoice line at its invoiced quantity and price	The invoice's own discount, unchanged
Credit	Specific lines	The ticked invoice lines, quantity and price as edited	Percentage: the same percentage. Fixed: invoice discount × (chosen lines ÷ all lines, after line discounts), half up at 3 decimals, capped at the credited amount
Credit	Custom amount	The note's own free lines only (start: one service line, unit `EA`, quantity 1, VAT category of invoice line 1). Several lines allowed for mixed VAT rates	`{FIXED_AMOUNT, 0}`
Debit	Correct existing lines (default)	Invoice lines whose extra price is above 0 and quantity above 0; each starts at price 0.000	`{FIXED_AMOUNT, 0}`
Debit	Additional items	The note's own lines only (start: one goods line)	`{FIXED_AMOUNT, 0}`

Note	Mode	`lines[]` sent	`headerDiscount` sent
Debit	Flat charge	Exactly one own line, a service (freight, late fee)	`{FIXED_AMOUNT, 0}`

Line fields: `lineNumber`, `productId?`, `productCode?`, `description`, `unitCode`, `quantity`, `unitPrice` (excl. VAT), `discount {type, value}`, `taxCategory {code, en16931Category, rate, exemptionReason, exemptionReasonCode}`, `classification?`.

- A line from the invoice keeps the invoice line's `productId`, `productCode`, unit, discount, VAT category, reason and classification. A line added on the note leaves `productId` and `productCode` out.
- Added lines: Goods/Service switch, units ending in ZZ Other, VAT category S5, Z0, E0 or O0 (default: the invoice's first line). Z needs a VATZR-OM code, E a VATEX-OM code, O optional free text, S nothing.
- `classification`: goods `{goodsOrService: "G", hsCode}` only when 12 digits; services `{goodsOrService: "S"}`, plus `serviceTypeCode` when 8 digits. Hidden for a consumer customer.
- Same HS search (`GET /v1/code-lists/hs`), service types and reason lists as invoices.

Rules to check before save (the server checks them too):

1. A reason is chosen: "Choose why the note is issued."
2. At least one line is included. Debit, Correct existing lines: "Enter the difference on at least one line: a price above zero and a quantity."
3. Quantity above 0, at most 12 digits and 6 decimals; price 0 or more, at most 15 digits and 6 decimals.
4. Credit only: a line from the invoice never above the invoiced quantity or unit price. The server also refuses a credit above what is still creditable.
5. Every line has a description.
6. Added Z and E lines carry their reason code.
7. One reason per VAT category and rate (BR-CO-18).
8. Issue date not in the future (Muscat time) and in the current year; debit due date not before it.

Totals are worked out on the device with the invoice math (`src/lib/invoiceMath.ts`): line discount, then document discount, then VAT per rate, three decimals half up. Show credit amounts as − in red and debit amounts as + in green.

4. List and summary

One list call and one summary call per kind; only `type` changes between credit and debit.

#	Call	Notes
4.1	`GET /v1/authored-invoices?type=CREDIT_NOTE&state=&payment=&from=&to=&q=&sort=&page=&size=` (or `type=DEBIT_NOTE`)	`sort`: `UPDATED` (default), `ISSUED_NEWEST`, `ISSUED_OLDEST`, `AMOUNT_HIGHEST`, `AMOUNT_LOWEST`, `CUSTOMER`. Pages 0-based; the web offers size 10, 25, 50
4.2	`GET /v1/authored-invoices/summary?type=CREDIT_NOTE` (or `DEBIT_NOTE`)	Counts and card figures, below
4.3	`GET /v1/branches`	Branch name under the customer when the note has no reference

Tabs

Note	Tab	Query	Count from summary
Both	All	`state=DRAFT,ISSUED,CANCELLED`	`all`
Both	Drafts	`state=DRAFT`	`drafts`
Both	To send	`state=ISSUED`, keep rows whose `deliveryState` is not `SENT_MANUALLY` (or send `delivery=TO_SEND`)	`toSend`
Credit	Applied	`state=ISSUED`	`issuedTotal.count`
Credit	Refund due, Refunded	No request yet: empty state	none
Debit	Awaiting payment	`payment=AWAITING`	`awaiting`
Debit	Overdue	`payment=OVERDUE`	`overdue`

Note	Tab	Query	Count from summary
Debit	Paid	`payment=PAID`	`paid`

Cards. Credit: Credited · 30 days = `issuedLast30Days {count, total}`; Applied to invoices = `issuedTotal`; Refund due and Refunded read "Not available yet". Debit: Charged · 30 days = `issuedLast30Days`; Awaiting = `awaitingTotal`, `awaiting`; Collected = `paidTotal`, `paidInvoices`; Overdue = `overdueTotal`, `overdueInvoices`, days late = `asOf` − `oldestOverdueDueDate`. Subtitle = `issuedThisQuarter`. Any of these may be missing on an older server: show "Not available yet", without a rial sign.

Row fields: `id`, `version`, `invoiceNumber` ("Draft" when null), `details.correctionReasonCode` and `correctionReasonDetail` (reason label), `correctionOfInvoiceId` and `correctionOfInvoiceNumber` (opens the invoice), `customerName`, `details.subject` (else the branch name), `issueDate`, `totals.totalAmount`, `authoringState`, `deliveryState`, `dueDate`, `payment {state, overdue, daysOverdue, lastPaymentDate}`.

Status: Draft; not issued and not a draft = Cancelled; issued and not `SENT_MANUALLY` = To be sent; then credit = Applied; debit = Overdue if `payment.overdue`, Paid if `payment.state` is `PAID`, else Awaiting payment. A draft row opens the editor with `invoice.draft`, otherwise the detail. Discarding a draft from the list uses 2.5.

5. Detail

The detail reads the note, its balance, files, history and note, the customer, and the original invoice. All under `/v1/authored-invoices/{id}` unless named.

#	Call	Notes
5.1	`GET /v1/authored-invoices/{id}`	The note: `invoiceNumber`, `issueDate`, `dueDate`, `details`, `lines[]`, `totals`, `correctionOfInvoiceId`, `authoringState`, `deliveryState`, `version`
5.2	`GET …/receivable`	The note's own balance (debit: what is still owed on it)

#	Call	Notes
5.3	`GET …/attachments`, `GET …/attachments/{attachmentId}`	List and download. No upload once issued yet
5.4	`GET …/history`	`{entries[], truncated, limit}` for Activity
5.5	`GET …/internal-note`	Only with `invoice.internal-note.read`; every read is audited
5.6	`GET /v1/customers/{customerId}`	Only with `customer.read`. Show "VAT {vatNumber} · 0248:…" built from `peppolParticipantId` without adding `0248:` twice
5.7	`GET /v1/authored-invoices/{correctionOfInvoiceId}` and its `/receivable`	The invoice the note corrects and its balance
5.8	`GET …/payments`	Debit notes only, not drafts
5.9	`POST …/payments`	Debit only. Header `Idempotency-Key: payment-<uuid>`. Body `{amount, methodCode, paymentReference, paymentDate}`. Needs `invoice.payment.manage`; only when `paymentState` is `UNPAID` or `PARTIALLY_PAID`
5.10	`POST …/payments/{paymentId}/reverse`	Debit only. `Idempotency-Key`, no body. Needs `invoice.payment.manage`
5.11	`GET …/reminders`, `GET …/reminders/preview?days_from_due=`, `POST …/reminders/send` with `{daysFromDue?}`	Debit only, awaiting or overdue. Sending needs `invoice.payment.manage`
5.12	`POST …/cancel`	`{expectedVersion, reason}`, reason required, max 1,000. Needs `invoice.cancel`; issued and not yet delivered only

Credit note figures. Credit total = `totals.totalAmount`. Only when this is the invoice's sole credit note (the invoice's `creditedAmount` equals this note's total): overpaid = max(0, invoice paid + invoice credited − invoice total), refund due = min(total, overpaid), applied = total − refund. With several credit notes on the invoice, Applied and Refund due read "Not available yet".

Debit note figures. Debit total = `totals.totalAmount`; now due on the invoice = invoice outstanding + this note's outstanding.

Not available yet (backend): there is no refund endpoint, so "Record refund" only says it is not available. There is no Peppol send, so notes stay "To be sent"; the web does not call `POST …/manual-delivery` for notes. There is no PDF endpoint.

6. Error codes and checklist

Note codes from create, update and issue. Issue refusals that apply to invoices too (dates, verification, quota, Peppol, ISIC, products, `STALE_INVOICE_VERSION`) are the same as in the invoice API doc, section 9. Payment refusals are the same as its section 7.

Code	What to tell the user
`CORRECTED_INVOICE_NOT_ISSUED`, `CREDIT_NOTE_REFERENCE_REQUIRED`, `DEBIT_NOTE_REFERENCE_REQUIRED`	This invoice is not issued (or was cancelled), so it cannot be corrected
`CORRECTION_REASON_CODE_REQUIRED`, `CORRECTION_REASON_CODE_INVALID`	Choose why the note is issued
`CORRECTION_REASON_DETAIL_CODE_MISMATCH`, `CORRECTION_REASON_DETAIL_INVALID`, `CORRECTION_REASON_DETAIL_NOT_ALLOWED_FOR_TYPE`, `CORRECTION_REASON_DETAIL_REQUIRES_CODE`	Choose why the note is issued
`CREDIT_NOTE_EXCEEDS_CORRECTABLE_AMOUNT`	This credit is more than is left to credit on the invoice
`CREDIT_NOTE_NOTHING_CORRECTABLE`	Nothing is left to credit on this invoice
`TAX_CATEGORY_REASON_CONFLICT`, `TAX_CATEGORY_METADATA_CONFLICT`	Lines at the same VAT category and rate must give the same reason

Code	What to tell the user
`HEADER_DISCOUNT_EXCEEDS_BASE`	The document discount is more than the note (avoid it with the cap in section 3)
`BUYER_REFERENCES_NOT_SELF_BILLED`	Never sent: leave `buyerReference` and `contractReference` out
`INVOICE_ATTACHMENT_REQUIRES_DRAFT`	Files can be added only while the note is a draft

Cancel (5.12)

Code	What to tell the user
`STALE_INVOICE_VERSION`	This note changed elsewhere; reload it and try again
`INVOICE_NOT_CANCELLABLE`	This note can no longer be cancelled
`DELIVERED_INVOICE_CANCELLATION_NOT_APPROVED`	A note that was already delivered cannot be cancelled

The English and Arabic wording is in `src/i18n/locales/en.json` and `ar.json` (keys `salesNote.*`).

Open questions for the backend

1. "Full invoice" on an invoice that already has a credit note credits the whole invoice; the server refuses it at issue if it is too much. Should mobile cap it to the creditable figure up front? The owner has not decided.
2. A refund endpoint and per-note refund figures, for the Refund due and Refunded tabs.
3. A reference field of its own: "Reference / RMA" is sent as `details.subject` today.

Checklist

- [ ] Notes start only from an issued invoice, with `creditnote.create` or `debitnote.create`
- [ ] Create is not wrapped; update is wrapped in `{expectedVersion, draft}`
- [ ] Reason sends the code and the reason id

- [ ] Credit modes Full invoice, Specific lines, Custom amount; debit modes Correct existing lines, Additional items, Flat charge
- [ ] Credit quantity and price never above the invoice's
- [ ] Document discount copied or shared out as in section 3
- [ ] One reason per VAT category and rate
- [ ] Save, upload files, then issue; keys reused on retry
- [ ] Credit due date = issue date; debit due date from the invoice's terms
- [ ] List tabs, cards and statuses as in section 4
- [ ] Debit: record and reverse payments, reminders
- [ ] English and Arabic for every label; three decimals with the rial sign